



DevOps in Software Engineering

Lecturer: Adel Vahdati



DevOps: From Code to Customer

- Scrum, Kanban, and Lean
 - Focus on how teams plan, organize, and build software.
- DevOps answers the question:
 - What happens *after* the code is written?
- DevOps looks beyond development to the entire path:
 - From a code change to a running system used by real users.
- It is **not** a single role or a collection of tools



DevOps: From Code to Customer

- DevOps is
 - a **mindset** that **encourages shared responsibility**,
 - a **set of practices** that **enable fast** and **reliable delivery**, and
 - a **system-wide approach** to **improving** how software is **built, deployed, operated, and learned** from across the organization.

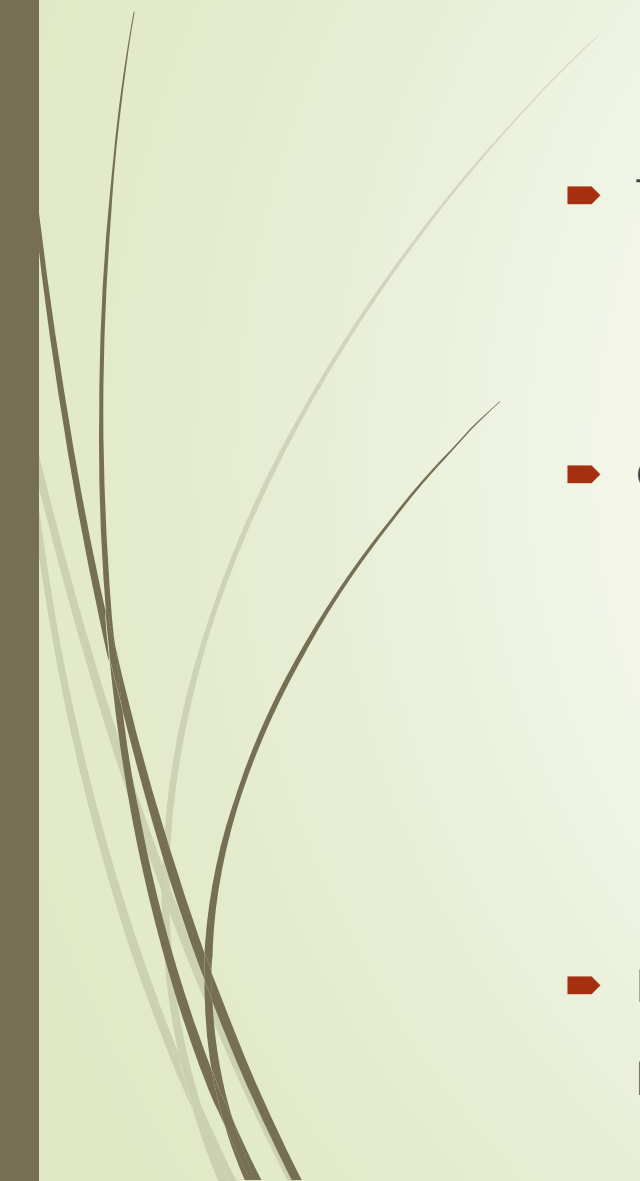


The Traditional Problem

- Traditional delivery model:
 - Developers write code
 - Operations deploy and maintain it
 - Separate teams, separate responsibilities
- This model worked when:
 - Software was delivered once or twice a year
 - Requirements changed slowly
 - systems ran on a small number of physical servers
- Problems arise when:
 - Change becomes frequent
 - Systems become complex



The Traditional Problem



- The core problem is the creation of **organizational silos**
 - teams optimize their own work locally but lose visibility and responsibility for the overall delivery system.
- Common symptoms:
 - Deployment failures
 - Long release cycles
 - Blame between teams
 - Fear of releasing changes
- Deployments fail unexpectedly, releases take a long time to prepare, and teams become afraid of making changes because failures are costly and highly visible.

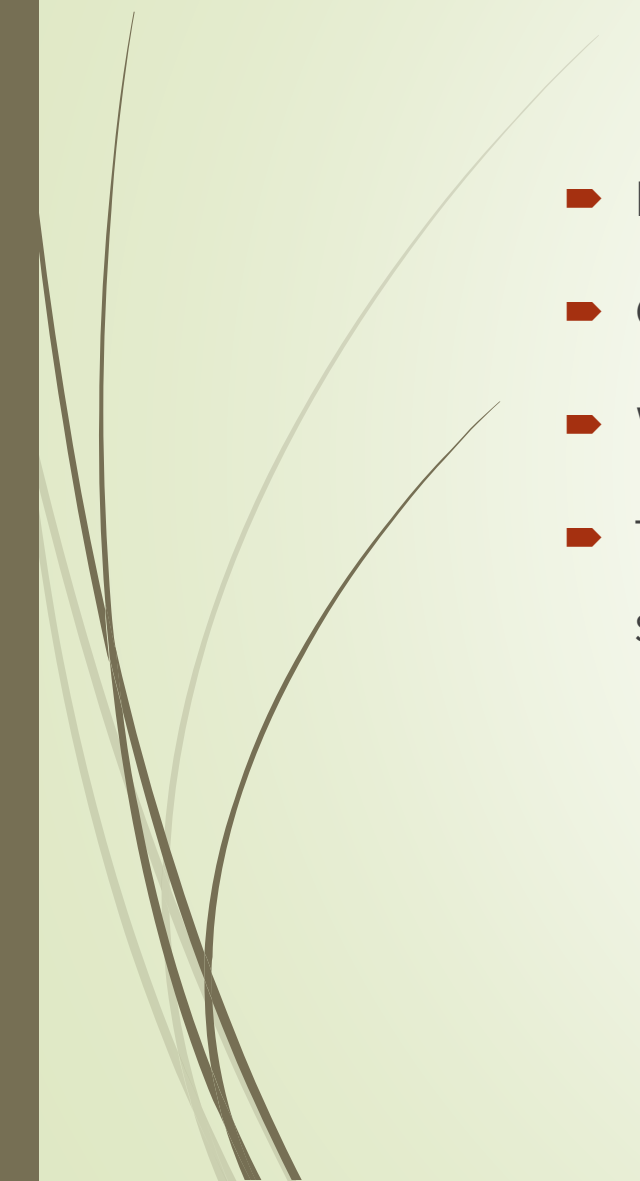


The Traditional Problem

- It Works on My Machine
 - Developer tests locally → works
 - Ops deploys → fails
 - Result: **Dev**: “Ops broke it” , **Ops**: “Dev gave us bad code”
- Goal conflict:
 - Developers → speed & features
 - Operations → stability & uptime
- This tension is *systemic*, not personal



The Traditional Problem

- Developers are rewarded for delivering new features quickly
 - Operations are rewarded for keeping systems stable and avoiding outages.
 - Without shared responsibility and collaboration, these goals collide.
 - This tension is systemic, meaning it is caused by how the organization is structured, not by individual incompetence or bad intentions.
- 



Why Agile Was Not Enough

- ▶ Agile improved:
 - ▶ Collaboration
 - ▶ Feedback
 - ▶ Adaptability
- ▶ But often: “Done” meant *code complete*
- ▶ Agile teams deliver increments But increments may:
 - ▶ Sit unreleased
 - ▶ Require manual deployment
 - ▶ Depend on Ops availability



What Is DevOps?

- A set of practices and principles that:
 - Integrate development and operations
 - Improve collaboration
 - Enable fast, reliable delivery
- It focuses on how work flows across the entire organization, from writing code to running systems in production.
- At its core, DevOps describes how the whole organization works together to deliver and operate software effectively.



Common Misconceptions

- DevOps is not a single job title such as a “DevOps engineer”
- it is not a separate DevOps department
- it is not simply a collection of automation tools
 - While tools and automation are important, they are only enablers.



Core Goal of DevOps

- DevOps aims to:
 - **Reduce lead time**
 - organizations can move from an idea or code change to a running feature in production much faster.
 - **Increase deployment frequency**
 - allows teams to release small, incremental changes instead of large, risky releases
 - **Improve reliability**
 - ensures that these frequent releases do not compromise system stability.
 - **Enable fast feedback**
 - leads to better technical and business decisions.



Core Goal of DevOps

- From a business perspective:
 - Faster feedback → better decisions
 - Reliable releases → trust
 - Short lead time → competitive advantage
- Faster feedback supports adaptation
- Reliable releases build trust with users and stakeholders
- Short lead times create a strong competitive advantage
- DevOps does not optimize individual teams in isolation; it focuses on optimizing the **entire delivery system** from code to customer.



DevOps and Lean Thinking

- DevOps applies Lean principles to:
 - Deployment
 - Infrastructure
 - Operations
- Reinforce Lean concepts:
 - Reduce handoffs
 - Remove waiting
 - Improve flow
- DevOps is Lean thinking applied to the software delivery pipeline



The CALMS Framework

- **DevOps consists of five essential elements:**
 - Culture
 - Automation
 - Lean
 - Measurement
 - Sharing
- **culture comes first, tools come last:**
 - Automation without culture → fragile systems
 - Measurement without trust → gaming metrics



The CALMS Framework

- ▶ Without a **culture** of **trust**, **collaboration**, and **shared responsibility**, automation alone tends to create fragile systems that break easily and are poorly understood.
- ▶ Measurement without trust can lead to people gaming metrics rather than genuinely improving the system.
- ▶ When **culture** is strong,
 - ▶ **Automation** supports reliability,
 - ▶ **Lean** principles guide flow,
 - ▶ **Measurement** enables learning, and
 - ▶ **Sharing** ensures knowledge is spread across teams.



DevOps Principles in Detail - Culture

- **Shared responsibility**

- Dev and Ops own outcomes together
- No throwing work over the wall

- **Collaboration**

- **Blameless learning**

- Failures are system problems
- Focus on learning, not punishment
- When failures occur, the focus is not on assigning fault to individuals, but on understanding how the system allowed the failure to happen and how it can be improved.
- This mindset encourages openness and continuous improvement.

- **“You build it, you run it.”**

- Those who create software are also responsible for its behavior in production.



DevOps Principles in Detail -Automation

- ▶ Automate:
 - ▶ Builds
 - ▶ Tests
 - ▶ Deployments
 - ▶ Infrastructure
- ▶ Why Automation Matters
 - ▶ Humans are naturally inconsistent, prone to error, and slow when performing repetitive tasks
 - ▶ Computers, on the other hand, execute the same instructions reliably every time.



DevOps Principles in Detail -Automation

- Automation enables:
 - **Speed:** allowing frequent changes to move quickly through the pipeline
 - **Reliability:** reducing the risk of errors during builds, tests, or deployments
 - **Confidence in change:** automated validation ensures that changes do not break existing functionality



DevOps Principles in Detail - Lean

- Small batch delivery enables faster feedback, safer deployments, and continuous learning.
 - **Fast feedback:** you receive feedback frequently, can adjust quickly
 - **Less waste:** fewer defects and less rework occur
 - **Easier to test:** each change is limited in scope
 - **Easier to rollback:** fewer components are affected if something goes wrong
 - **Easier to understand:** developers and operations can clearly see what each change does.



DevOps Principles in Detail - Measurement

- Measure:
 - **Lead time:** how long it takes a change to reach production
 - **Deployment frequency:** how often changes are released
 - **Failure rate:** how often deployments cause issues
 - **Recovery time:** how quickly problems are resolved
- Metrics should **measure the system and processes**, not individual performance.
- Good metrics guide teams in making improvements, identifying bottlenecks, and optimizing flow
- Poorly chosen metrics can create perverse incentives, such as
 - rushing releases to hit a number
 - blaming individuals instead of fixing systemic issues.
- The goal is to foster learning and continuous improvement, not punishment.



DevOps Principles in Detail - Sharing

- In traditional organizations, silos form when knowledge is locked within individual teams or roles,
 - making it hard to see problems until they become critical.
 - this lack of transparency can hide inefficiencies, defects, or bottlenecks.
- DevOps counters this by promoting:
 - Transparency
 - Knowledge sharing
 - Cross-team learning
- Examples include
 - shared dashboards that show deployment and system status,
 - shared documentation of processes and infrastructure,
 - shared responsibility for outcomes.



Continuous Integration (CI)

- **Frequent code integration:** CI, is a practice where developers integrate their code into a shared codebase frequently, often multiple times a day.
- **Automated builds and tests:** Each integration triggers automated builds and tests, which immediately verify that the new changes work correctly with the existing system.
 - Early defect detection
- Without CI, teams often experience “**integration hell**,” where code is merged only at the end of a development cycle.
 - Many conflicts
 - Hard to debug
- CI solves this by:
 - Encouraging **small, frequent integrations**,
 - Making issues easier to detect, isolate, and fix early, when they are cheaper and less risky to resolve.



Continuous Delivery (CD)

- Continuous Delivery, or CD, is the practice of ensuring that software is always in a deployable state.
- **Software always deployable:** at any point in time, the system has passed the necessary tests and checks and could be released to users if the organization chooses to do so.
- Releases are routine and low-risk
- Continuous **delivery** → ready to deploy
- Continuous **deployment** → automatically deployed
- The key idea is **deployability**, not automation alone.
 - making releases routine, predictable, and low-risk rather than rare, stressful events.



Deployment Pipeline

- Typical stages:
 - Commit
 - Build
 - Test
 - Deploy
 - Monitor
- Each stage adds confidence that the change is correct and safe.
- Detecting a problem early in the pipeline (e.g. during build or testing) is relatively cheap and easy to fix.
- In contrast, failures discovered late, especially in production, are much more expensive and risky.
- The goal of a well-designed pipeline is to provide fast feedback and catch problems as early as possible.



Infrastructure as Code (IaC)

- IaC, is the practice of defining and managing infrastructure (such as servers, networks, and configurations) using code rather than manual setup.
- IaC treats infrastructure like software.
- Version-controlled
 - making infrastructure changes traceable, reviewable, and repeatable.



Automated Testing

- Unit tests
- Integration tests
- Acceptance tests
- **testing pyramid:**
 - many fast, inexpensive unit tests at the base;
 - fewer integration tests in the middle to verify interactions between components;
 - small number of acceptance tests at the top to confirm end-to-end behavior from the user's perspective.
- **Key idea:**
 - Automated tests make change safe.
 - Manual testing alone cannot keep up with fast delivery.



Monitoring and Feedback

- Monitoring and feedback focus on observing how systems behave in real production environments. This includes:
 - **collecting metrics** (such as performance and availability),
 - **logs** (detailed records of system behavior),
 - **alerts** (notifications when something goes wrong).
- Allowing teams to **learn from actual usage**, **detect problems quickly**, and **continuously improve** both the system and the delivery process.